

What the Erroneous Hawaiian Missile Alert Can Teach Us About Error Prevention

by [Kim Flaherty](#) on January 16, 2018

Topics:

- [Application Design](#)

Summary: People should not be blamed for errors caused by poorly designed systems.

On Saturday, January 13th 2018, residents and tourists in the Hawaiian islands received a very frightening emergency alert on their mobile devices. The alert from Hawaii's emergency alert command center read "BALLISTIC MISSILE THREAT INBOUND TO HAWAII. SEEK IMMEDIATE SHELTER. THIS IS NOT A DRILL."



The missile emergency alert was sent to residents and tourists in Hawaii. Note the all-capitals font: while capital letters can be appropriate for [glanceable text](#), a 3-line paragraph is difficult to read when formatted like this.

Thankfully, this was in fact a drill: there was no inbound missile headed toward the islands. However, the message had been inadvertently sent live to cell phones across the islands rather than to the test network it had been meant for. Since this incident occurred, there has been extensive news coverage about the operator who was responsible for this false alarm. Many headlines focused on the human error, and some even called for the job of the person responsible:

- *Hawaii missile alert: How one employee 'pushed the wrong button' and caused a wave of panic* – Washington Post
- *'Wrong button' sent false missile alert to Hawaii* – Daily Beast

- *'Inexcusable' false ballistic missile alert in Hawaii was caused by human error* – TechCrunch

Apparently, the person erred twice:

1. by selecting the wrong choice (real alert instead of test)
2. by confirming the previous selection

This angle to the story did not sit well with me. When I heard about this mishap, my immediate question was, “What is wrong with the interface that caused the operator to inadvertently make the incorrect selection?”

Slips and Error Prevention

I think we can all relate to pushing the wrong button by accident. I’ve cancelled transactions at store checkouts and gas pumps after pushing *No* instead of *Yes*. I’ve opted into add-on offers on websites by inadvertently selecting *Accept* rather than *Decline*. Luckily none of these mistakes sent an entire state into panic, but they are all examples of [slips](#).

Slips occur when a user wants to do one action but unintentionally takes another (usually similar) action. They often happen in situations where users are on autopilot and do not fully devote their attentional resources to the task at hand. Accidentally putting liquid hand soap on one’s toothbrush instead of toothpaste is an unconscious slip. But slips can also happen when the two choices look too similar or are placed too closely to each other.

Often times, when you stop to assess the design of an interface where a slip has occurred, there is an underlying UI issue at play that sets people up for these errors.

There are a few common system-design problems that cause unconscious slips:

- **Miscues:** When the design gives the wrong cue to users — for example, by violating design conventions
- **Poorly differentiated options:** When an interface provides multiple choices, but it uses similar names or similar visual styling for them
- **Problematic presentation or interaction design:** When the layout, presentation, or workflow does not include appropriate safeguards — for example, by placing two buttons very close to each other
- **Lack of confirmation for destructive actions:** When the design does not ask users to [confirm an action with major consequences](#)

All these issues illustrate poor error prevention. Error prevention is one of Jakob Nielsen’s [10 heuristics for user interface design](#); it becomes especially important for critical systems like the Hawaii emergency-alert system or hospital systems where pushing the wrong button could be the difference between life and death.

In the case of the mistaken missile alert, some news outlets have provided examples of the interface involved. The same system screen is used to deliver several different types of emergency alerts to the public including amber alerts, road closures, and high-surf warnings. There are 9 unique options which include both real-time alerts that go to the general public and drill versions of these alerts meant to test and troubleshoot them. (A 10th option was added later, to send a “it was a false alarm” retraction message in case of future errors.)

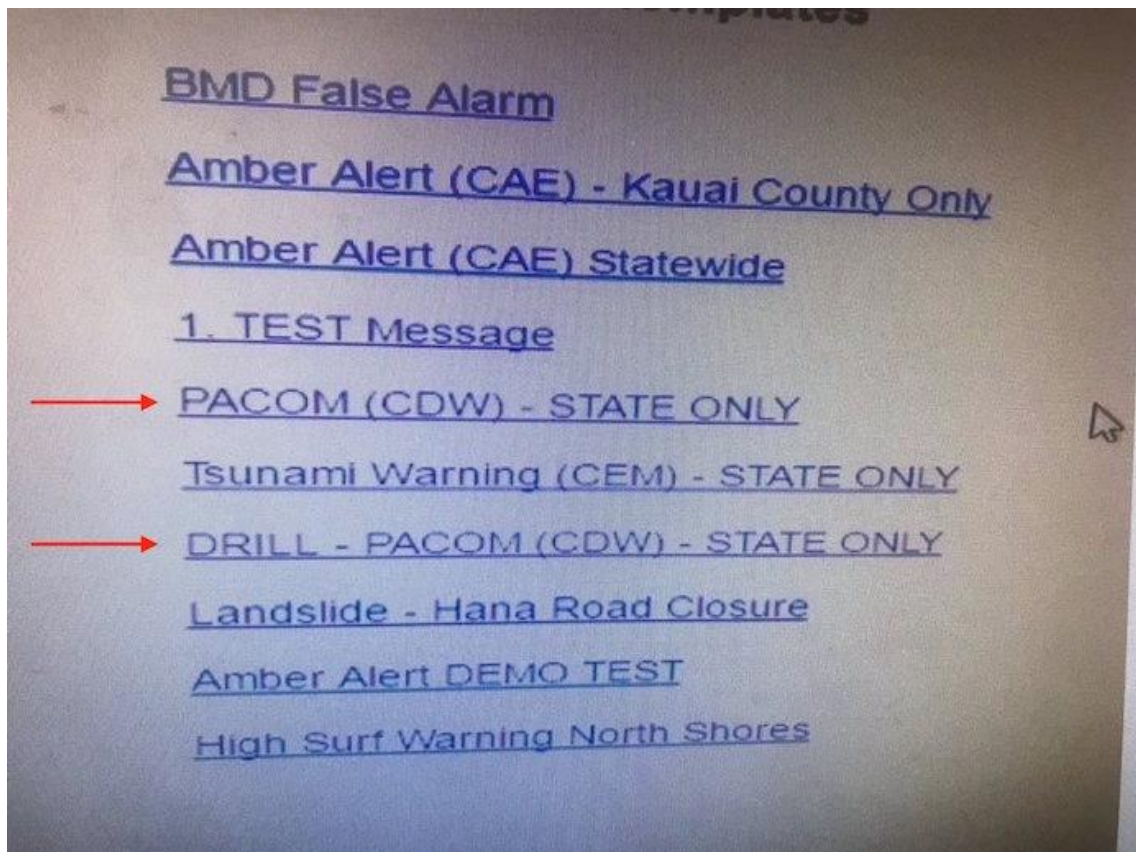
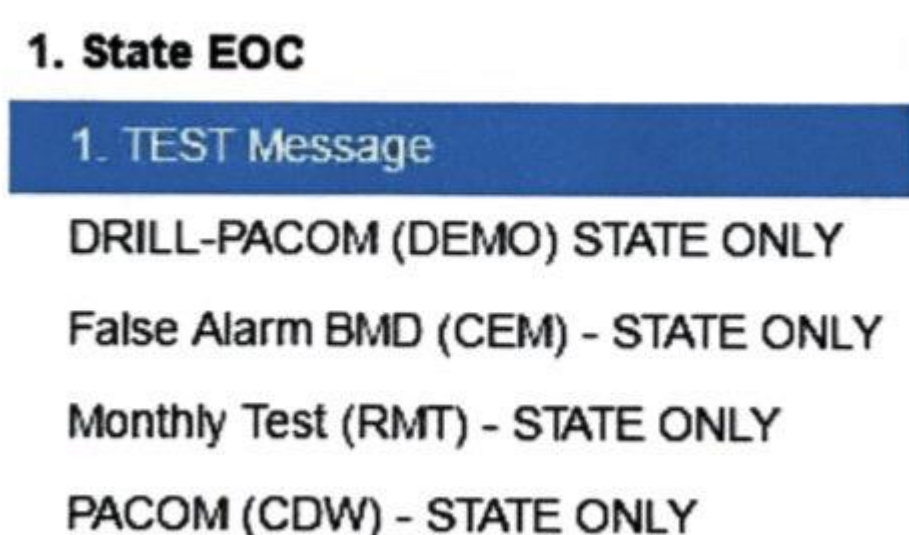


Photo released by the Hawaii Governor's office of the system screen that was used to send out the erroneous missile alert. (Note that the first option in the list, BMD False Alarm, was added after the mishap, to allow for sending retraction messages in the case of future false alarms. Normal [menu-design guidelines](#) would have recommended putting this option at the bottom of the list, since it's presumably a rarely used choice.) Source: [Honolulu Civil Beat](#)



Revised screenshot released by the Hawaii Emergency Management Agency after we wrote this article. Supposedly, the Governor's screenshot may not have been the one actually in use. This second design suffers from the same usability problems, but is marginally better in having a smaller number of obscure and poorly formatted menu options.

The two options that were involved in this mishap were the following:

- PACOM (CDW) – STATE ONLY : the option the operator mistakenly selected
- DRILL- PACOM (CDW) – STATE ONLY : the option the operator should have selected

The operator was supposed to have selected the second option DRILL-PACOM (CDW) – STATE ONLY, but instead selected the other one.

Interface Design Problems

This missile-alert UI includes several design problems:

- **Poorly differentiated options:** The options listed on the page have cryptic labels that require too much mental effort to differentiate. Moreover, some of these labels (for example, the two relevant to this incident) are fairly similar. Couldn't these options be replaced with simpler ones like those below?

- Test Alert: PACOM
- Live Alert: PACOM

One may argue that these labels are not descriptive enough on a screen that contains many different types of alerts that are listed in no particular order. Why not group them in a meaningful way – for example, with all amber alerts placed together under a heading meant to distinguish them?

- **Problematic presentation or interaction design:** Maybe the root of the problem comes before this screen entirely. Should we really have the live vs. test options presented in the same environment and on the same screen for critical system like this? With this design, a slip of the finger could result in a major problem, as seen. Why not have two separate workflows or modes? One [mode reserved for test work](#), which should be the default, and another for live system commands, where an explicit action must be taken to enter and submit commands. In my days as a developer, we had such structural safeguards in place, with live environments completely separated and visually distinct from the test environments.
- **Possibly poor confirmation screen:** News reports point out that, after the operator selected the wrong network, she was presented with a confirmation screen — indicating that the system's designers may have tried to create some safeguards against accidental actions. But what did that screen look like? Often, when people go to similar confirmation screens ten times a day or when every action has a confirmation screen, they no longer pay attention to them — these dialogue boxes feel like roadblocks in their flow.

Could a more intrusive type of confirmation for a live alert have helped? Perhaps it could have required users to reenter their passwords — not for added security against intruders, but to make it clear to users that they were about to do something unusual with serious consequences. Many banks use such an intrusive confirmation step before initiating irreversible actions such as a wire transfer.

Lack of Undo Capability

Another application-design heuristic violated by the Hawaiian warning system is the ability to undo an erroneous command. The sending of messages cannot literally be undone, which is why a confirmation step is appropriate before the system starts sending. But one can at least send out a second message saying that the first one was a false alarm. This was indeed done, but only after a 38-minute delay.

According to news outlets, the delay was caused by bureaucratic disagreements between different levels of government, because the Hawaiian state agency that issued the (false) alert was not authorized to send anything except alerts. The agency had to secure additional levels of approval in order to send out a second, different type of message. However, whoever is authorized to take a certain action ought also to be authorized to undo that action if it turned out to be a mistake. In addition to this bureaucratic problem, sending a message to rescind the prior alert was not supported by the system, and this capability had to be created in the system on-the-fly by off-site developers.

Mistakes will be made, so plan for them. (Of course, the exact same mistake will likely not happen, but other mistakes will. This is why systems need to be designed to alleviate mistakes.)

Conclusion

The takeaway that human error was to blame and the user was at fault for having selected the incorrect option is wrong. Altogether **a poorly designed system is to blame**. A good UI makes it hard for people to err, and easier to recover from any remaining errors.

The solution is not to scold users, ask them to try harder, or to give them more extensive training. The answer is to redesign the system to be less error-prone.

Let's just be happy we discovered this design problem now. Imagine if there was a real threat and the user accidentally sent the message to the test network rather than the live network.